

Article

Development and Optimisation of a Modular Open-Source System for Real-Time Remote Video Production

Martín Herranz-Sánchez ¹, Álvaro Llorente-Gómez ^{1,*}, David Jiménez-Bermejo ¹ and Alberto del Río-Ponce ¹

¹ Departamento de Señales, Sistemas y Radiocomunicaciones, Escuela Técnica Superior de Ingenieros de Telecomunicación, Universidad Politécnica de Madrid, 28040 Madrid, Spain; martin.herranz@alumnos.upm.es (M.H.S.); alvaro.llorente@upm.es (Á.L.); david.jimenez@upm.es (D.J.B.); alberto.delrio@upm.es (A.d.R.P.)

* Correspondence: alvaro.llorente@upm.es

Abstract

Professional live video production has historically depended on dedicated hardware switchers costing hundreds of thousands of dollars, placing broadcast-quality production out of reach for research groups and small broadcasters. Software alternatives such as OBS Studio or vMix lower the financial barrier but remain monolithic single-operator applications that cannot run headlessly, expose no machine-readable control API, and lack native container support. This paper presents Voctomix 2.0, an extended and fully containerised version of the open-source Voctomix live video mixer, built on GStreamer and validated across four deployment scenarios from a single workstation to a Kubernetes cluster. Its contributions are a real-time overlay and dual-text titling system with automatic deactivation on cuts, a three-state stream blanker (LIVE / PAUSE / NOSTREAM), Audio Follows Video cross-fading, a RabbitMQ AMQP telemetry channel exporting the full mixer state without core changes, and a ten-container Docker Compose stack deployable with a single command. On four 1080p25 sources, the Docker Compose deployment runs at 38.8% CPU, with source-switching latency below 5 ms (median 1.5 ms), far under the 45 ms ITU-R BT.1359-1 threshold. The system runs for 31 minutes with no memory leak and recovers a failed camera container in about 520 ms. A production deployment within the CyberNEMO European research project at Universidad Politécnica de Madrid confirms operational readiness beyond the laboratory.

Keywords: GStreamer; live video production; remote production; open-source; Docker; Kubernetes; video mixing; broadcast automation; real-time

1. Introduction

The broadcast industry's transition from dedicated Serial Digital Interface (SDI) [1] infrastructure to IP-based workflows has dramatically lowered the cost of signal transport, yet the mixing and compositing layer has remained anchored to proprietary hardware. A complete production unit based on hardware switchers such as the Blackmagic ATEM can still exceed \$2 million, demands specialised on-site personnel for installation and operation [2], and ties organisations to vendor-specific hardware refresh cycles. The Remote Integration Model (REMI) partially addresses the logistical dimension by concentrating mixing in a centralised IP-connected facility while cameras and contribution encoders remain on site, enabling production teams to work from any location with network connectivity [3].

Software-based tools have further lowered the entry point. OBS Studio provides free, open-source streaming and recording, but its monolithic design couples capture,



Received:

Revised:

Accepted:

Published:

Copyright: © 2026 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the [Creative Commons Attribution \(CC BY\)](https://creativecommons.org/licenses/by/4.0/) license.

compositing, and output in a single application for a single operator, with no machine-readable control interface suitable for server-side or multi-operator deployments [4]. vMix offers professional multi-source mixing with Network Device Interface (NDI) and 4K support, but its commercial licence (exceeding \$1000), mandatory Windows platform, and absence of a containerisation path exclude it from Linux server infrastructure and reproducible cloud deployments [5]. Neither tool exposes a structured network Application Programming Interface (API), operates headlessly on a remote server, nor integrates with standard container runtimes.

Voctomix, developed by the C3VOC team for the Chaos Communication Congress, introduced a client–server architecture that structurally decouples the multimedia mixing engine (voctocore) from the operator control interface (voctogui), enabling distributed production over standard TCP networks and built on the GStreamer multimedia framework [6,7]. Recent work demonstrated its viability in real operational conditions: del Río et al. deployed Voctomix v1.3 as the Production Core of a distributed live media pipeline during a large-scale sporting event, with operators controlling the mix from Madrid while the event took place in Athens [8]. That deployment confirmed the soundness of the client–server model but exposed operational gaps: the absence of a real-time telemetry channel, a manual dependency installation process that limits reproducibility, and a limited set of composite and overlay capabilities required for professional production workflows.

This paper presents Voctomix 2.0, an extended and fully containerised version of the system that closes those gaps. The principal contributions of this work are:

- A real-time overlay and dynamic lower-third system supporting PNG graphics and two simultaneous text layers, with configurable fade-in/fade-out and automatic deactivation on every camera cut.
- A three-state stream blanker (LIVE / PAUSE / NOSTREAM) integrated as a GStreamer pipeline module, allowing broadcast state management without interrupting the internal mixing process.
- An Audio Follows Video (AFV) mechanism that automatically cross-fades audio levels on source switches, eliminating manual audio management and reducing operator cognitive load during live production.
- A telemetry service that exports the complete mixer state as JSON and publishes change and periodic state events to a RabbitMQ Advanced Message Queuing Protocol (AMQP) broker, enabling integration with external monitoring systems without modifying the application core.
- A complete Docker Compose stack of ten containerised services, deployable with a single command, sustaining four video sources at 1920×1080 pixels, 25 fps with 38.8% CPU usage on a 10-core workstation.
- Validation across four deployment scenarios, from single-PC development to Kubernetes orchestration, plus a production deployment within the CyberNEMO European research project at Universidad Politécnica de Madrid (UPM).

The remainder of this paper is organised as follows: Section 2 reviews related work in live video production systems and containerised media workflows. Section 3 describes the Voctomix 2.0 system architecture. Section 4 presents the deployment scenarios. Section 5 reports validation results and performance metrics. Section 6 discusses the findings, limitations, and directions for future work. Section 7 presents the main conclusions.

2. Background and Related Work

2.1. Live Video Production Systems

Production switchers have traditionally been dedicated hardware units. The Black-magic ATEM Constellation 8K and comparable Ross Video systems require SDI matrix routing, specialised control surfaces, and on-site engineering teams, placing complete production suites in the \$500 k to \$2 million range [2]. These constraints exclude small-scale broadcasters, academic institutions, and research groups from professional live production.

Software mixers lower the financial barrier but introduce other limitations. OBS Studio provides free capture, compositing, and streaming, yet its monolithic process model couples all functions in a single application window designed for a single operator; it offers no machine-readable TCP or REST control interface and cannot run headlessly on a server [4]. vMix supports multi-source mixing, NDI input, and 4K output, but its commercial licence (from \$60 to over \$1 000 per seat), mandatory Windows platform, and absence of a container runtime path exclude it from Linux infrastructure and reproducible cloud deployments [5]. The Remote Integration Model (REMI) partially addresses the logistical barrier by concentrating mixing in a centralised IP-connected facility, but existing REMI implementations rely on the same proprietary hardware-software stacks [3,9]. Large-scale deployments confirm the viability of this approach: Olympic Broadcasting Services applied REMI workflows to the Tokyo 2020 Olympic Games [10], and Telefónica Servicios Audiovisuales, together with Appear, was selected to power LaLiga's new Secure Reliable Transport (SRT)-based contribution and distribution network operated from Madrid [11]. None of these solutions, however, expose a structured API for programmatic control, operate without a display server, or integrate natively with container orchestration platforms.

2.2. Open-Source Media Pipelines

GStreamer is a plugin-based multimedia framework that models media processing as directed graphs of elements connected by pads [7]. Its bus-based pipeline architecture, caps negotiation system, and support for hardware-accelerated elements (VA-API, NVENC) make it well suited to building modular, real-time processing graphs that can be inspected and modified at runtime through its introspection API. FFmpeg provides extensive codec support and format conversion capabilities but is oriented toward file-based or single-stream command-line workflows rather than multi-source live compositing.

The contribution path feeding a software mixer is equally critical. Professional broadcast has migrated from hardware SDI matrix routing under SMPTE ST 2022-6 [12] towards uncompressed IP media via SMPTE ST 2110 [13] and software-defined transports such as NDI [14] and SRT [15], reducing capital cost while preserving broadcast-grade reliability. The broader migration toward sub-second glass-to-glass latency in live media streaming has been surveyed in recent work [16]. Multi-source synchronisation in these IP environments depends on IEEE 1588 Precision Time Protocol (PTP) [17].

Voctomix, originally developed by the C3VOC team for the Chaos Communication Congress, was the first open-source system to apply a true client-server split to live video production: a GStreamer-based mixing engine (voctocore) decoupled from an operator GUI (voctogui) over a TCP control channel [6,18]. This architecture enables multiple remote operators to share a single mixer instance and allows the core to run on a dedicated headless server. del Río et al. demonstrated its operational viability by deploying Voctomix v1.3 as the Production Core of a quality-aware live streaming pipeline during a large-scale sporting event with geographically separated production teams [8]. That deployment validated the client-server model under real broadcast conditions and provided the baseline from which Voctomix 2.0 is derived.

2.3. Containerised Media Workflows

Containerisation via Docker addresses the reproducibility gap inherent in native media software stacks, where library version conflicts, codec dependencies, and driver requirements create fragile deployment environments. A Docker image encapsulates the full dependency graph, enabling identical execution across development laptops, on-premises servers, and cloud instances [19]. Docker Compose extends this to multi-service topologies by declaratively specifying inter-service networking and startup ordering, which is directly applicable to the producer–consumer model of a mixer with multiple camera ingestion services.

Kubernetes (K8s) adds horizontal scaling, self-healing, and declarative rollout management to containerised workloads. Real-time media imposes requirements—bounded scheduling latency, access to GPU or capture-card hardware, and display-less headless operation—that have historically complicated container deployments. Recent advances in device plugin frameworks, the `hostNetwork` option for latency-sensitive pods, and container-aware GStreamer builds have progressively reduced these barriers. Industry analysis confirms that cloud-native and containerised broadcast workflows are becoming mainstream [20], and academic evaluations have shown that containerised GStreamer pipelines on edge hardware achieve performance comparable to native execution [21]. Several broadcast vendors have begun offering cloud-native media processing tools (e.g., Blackmagic Cloud, AWS Elemental), but these are proprietary, consumption-priced services with no provision for on-premises deployment or source-level customisation. Voctomix 2.0 fills the gap between these commercial cloud offerings and the uncontainerised, single-server Voctomix v1.x, providing a fully open containerised production stack deployable from a single `docker compose up` command.

3. System Architecture

3.1. Architecture Overview

Voctomix 2.0 follows a strict client–server separation (Figure 1). The processing server, `votocore`, hosts a GStreamer multimedia pipeline that accepts up to six simultaneous video/audio input streams over TCP, performs compositing and mixing, and delivers the programme signal on port 15000. The operator interface, `votogui`, is a GIMP Toolkit (GTK) 3 desktop application that connects to `votocore` over a dedicated control channel on port 9999 and receives per-source JPEG preview streams on ports 14000–14005. Multiple `votogui` instances may connect simultaneously, enabling multi-operator workflows without restarting the mixing engine.

In the Docker Compose deployment, the complete stack—`votocore`, four FFmpeg camera-ingest containers, auxiliary source containers, a RabbitMQ broker [22], and a telemetry service—runs within a shared bridge network. Camera containers use `network_mode: service:votocore`, which places them in the same network namespace as `votocore` so that inter-container communication traverses the loopback interface, eliminating the overhead of a virtual Ethernet link. The entire stack launches with `docker compose up` [19].

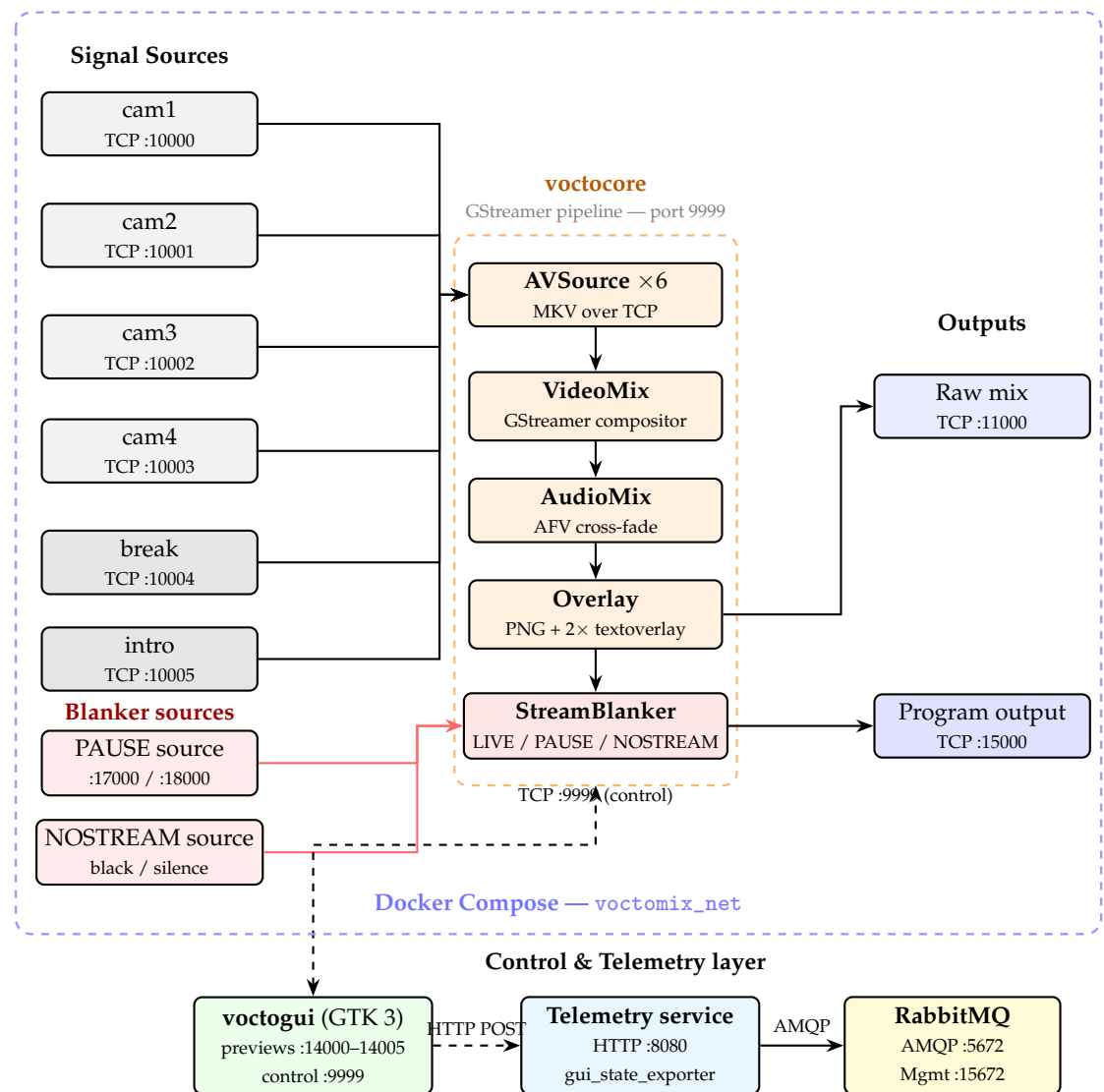


Figure 1. Voctomix 2.0 system architecture. Signal sources push Matroska-wrapped raw streams to voctocore via TCP. The GStreamer pipeline performs source selection, compositing, audio mixing, overlay injection, and output gating. voctogui controls the mixer over a line-based TCP protocol and exports state via the telemetry chain to RabbitMQ.

3.2. voctocore: the Processing Core

3.2.1. GStreamer Pipeline Design

voctocore constructs a single GStreamer pipeline at startup through `pipeline.py`, which instantiates and links all processing elements in the correct order [7]. The internal video format is 8-bit YUV (I420, 4:2:0) at 1920×1080 pixels and 25 fps; audio is 16-bit linear PCM (S16LE) at 48 kHz, stereo. These formats are negotiated at source boundaries via GStreamer caps and maintained throughout the pipeline without intermediate re-encoding, preserving signal fidelity and minimising CPU usage from format conversion.

Inter-module data flow uses GStreamer `appsrc/appsink` pairs, which allow each processing module (`videomix.py`, `audiomix.py`, `overlay.py`, `streamblanker.py`) to be implemented as an independent Python object that injects and extracts buffers without pipeline relinking at runtime. All GStreamer state transitions are confined to initialisation; the pipeline graph is static during operation.

A `NetTimeProvider` element broadcasts a GStreamer network clock over UDP port 9998. All camera-ingest containers synchronise to this clock, ensuring presentation-time alignment across all six input streams.

3.2.2. Source Management

Six logical source slots are defined: `cam1–cam4` for live camera feeds, `break` for a pre-recorded interstitial loop, and `intro` for an opening sequence. Each slot listens on a dedicated TCP port (10000–10005); the source plugin is selected via the `kind=` parameter in the configuration file, enabling per-slot switching between live, static, and hardware capture modes without modifying the pipeline code.

`tcpavsource.py` accepts a Matroska-wrapped [23] raw stream pushed by an upstream FFmpeg process. In the Docker deployment, one FFmpeg container per camera decodes the incoming video and re-muxes it into the Matroska-over-TCP format expected by `votocore`. `imgvsource.py` loops a static PNG or JPEG image as a source and is used for the `break` and `intro` slots in native deployments. A DeckLink capture-card source (`decklinkavsource.py`) is available for hardware SDI integration.

3.2.3. Composite Modes and Video Mixing

`videomix.py` maintains two logical channels, A and B, each assigned to one source slot. On each source-switch command, it updates the GStreamer compositor element's pad offsets, dimensions, and alpha values according to the parameters returned by `composites.py` for the active composite mode. Mode changes take effect at the next GStreamer buffer boundary, producing a clean hard cut with no visual artefacts.

Six composite modes are supported (Table 1). *Fullscreen A/B* renders a single source at full resolution. *Picture-in-Picture* (PIP) places source B in a configurable corner at 25% of frame width, over source A. *Side-by-Side Equal* divides the frame vertically between A and B at equal width. *Side-by-Side Preview* allocates 75% of the width to A and 25% to B. *Lecture* is optimised for slide-and-presenter productions with a large slide window (source B) and a small presenter overlay (source A). *Lecture 4:3* constrains the slide area to a 4:3 aspect ratio for legacy presentation content.

Table 1. Composite modes supported by `videomix.py`.

Mode	Description	Channel A	Channel B
Fullscreen A	Source A at full resolution	Full frame	Hidden
Fullscreen B	Source B at full resolution	Hidden	Full frame
PIP	Source B in corner (25% width) over source A	Background	Corner
SBS Equal	A and B at equal half-width	Left 50%	Right 50%
SBS Preview	A large, B small preview strip	Left 75%	Right 25%
Lecture	Large slide (B) + small presenter (A)	Overlay 25%	Slide 75%
Lecture 4:3	As Lecture, slide area constrained to 4:3	Overlay 25%	Slide 4:3

3.2.4. Audio Follows Video

`audiomix.py` implements Audio Follows Video (AFV): when the active source A changes, the module automatically schedules a cross-fade between the outgoing and incoming audio channels. The fade is realised by linearly interpolating the volume property of the corresponding GStreamer volume elements over a configurable duration (200 ms by default), driven by a pipeline clock callback. The outgoing channel fades from unity gain to silence while the incoming channel fades from silence to unity, producing a perceptually clean transition without clicks or pops.

AFV operates independently of the composite mode: switching to PIP or SBS does not suspend the mechanism, so audio always follows source A regardless of the visual layout. Operators retain the ability to override volumes manually through the GUI; AFV re-engages on the next source switch.

3.2.5. Output Control: Stream Blanker

The stream blanker controls what is delivered to the programme output port 15000, independently of the internal mixing state. `streamblanker.py` uses a GStreamer `input-selector` element to choose among three signal paths.

In LIVE state, the mixed programme signal passes through unmodified. In PAUSE state, the selector routes a dedicated PAUSE source (a static image loop on port 17000) to the output, with a background audio signal on port 18000, allowing hold music to play while the internal mix continues to evolve off-air. In NOSTREAM state, a silent black frame is delivered to port 15000, effectively cutting the broadcast output entirely.

State transitions occur at the GStreamer buffer boundary, producing clean hard cuts with no encoder artefacts. The blanker state is reflected in real time on the GUI toolbar and is included in every telemetry event.

3.2.6. Overlay System

The overlay system composites graphical lower-thirds and text annotations over the programme mix in real time. `overlay.py` in `votocore` manages a GStreamer `gdpxbufoverlay` element for PNG graphics and two independent `textoverlay` elements for text layers. PNG compositing uses the image alpha channel; the typical deployment renders a branded graphic as the lower-third background at a fixed screen position configurable in the INI file. Each text layer has separately configurable font, size, colour, and position; the first layer is typically used for a name and the second for a role or affiliation.

Fade-in and fade-out are implemented by linearly animating the alpha property of all overlay elements over 500 ms, driven by the GStreamer pipeline clock. Crucially, the overlay is automatically deactivated on every source cut: whenever `set_video_a` or `set_composite_mode` is received, `votogui/lib/toolbar/overlay.py` issues `show_overlay false` before forwarding the original command, ensuring lower-thirds never persist across unintended cuts.

3.3. *votogui: the Control Interface*

`votogui` is a GTK 3 desktop application that provides the operator interface for the mixing session. It runs an async event loop alongside the GTK main loop, using non-blocking TCP I/O for all communication with `votocore`, so the interface remains responsive under network latency.

The window is divided into three panels (Figure 2). The left panel displays five JPEG preview thumbnails—one per active source (cam1–cam4) plus the current mix output—received on ports 14000–14005 and refreshed continuously at the `votocore` output frame-rate. The right panel shows the full-resolution programme monitor feed. The central toolbar provides: source A and source B selector buttons (one per configured source slot); a composite mode drop-down listing all supported modes; LIVE/PAUSE/NOSTREAM blanker buttons with colour-coded state feedback; and an overlay control panel with two text input fields, a PNG path selector, and an on/off toggle with fade control.

The `gui_state_exporter.py` module runs a one-second GLib timer that captures the complete mixer state as a JSON object (active sources, composite mode, blanker state, overlay text fields, audio levels) and writes it to `registros/gui_state.json` in native mode, or HTTP-POSTs it to the telemetry service in Docker mode.

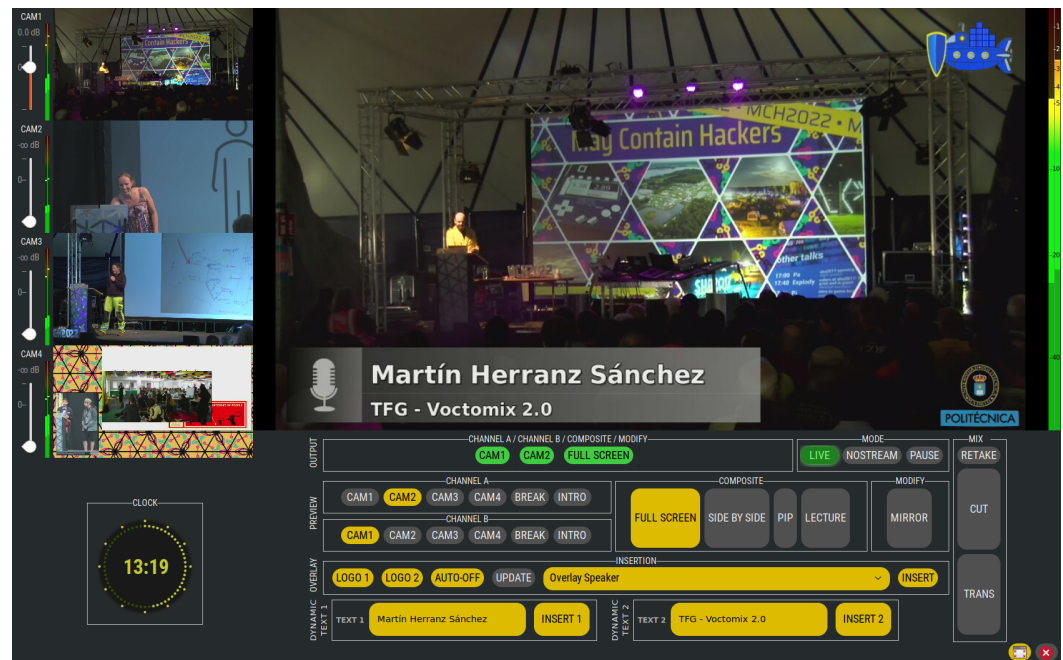


Figure 2. The vactogui operator interface. The left panel shows four live source previews and the current mix output. The right panel displays the full-resolution programme monitor. The toolbar provides source selection (A/B buttons), composite mode control, stream blanker state (LIVE/PAUSE/NOSTREAM), and overlay management.

3.4. TCP Control Protocol

All mixer control is carried over a line-delimited ASCII protocol on TCP port 9999, implemented in `commands.py` and `controlserver.py`. Each command is a single UTF-8 line terminated by a newline character. The server validates the command, executes the corresponding pipeline action, and broadcasts a response event of the same form to *all* connected clients, ensuring that multiple simultaneous vactogui instances maintain a consistent view of the mixer state without a separate synchronisation mechanism.

The core command set is as follows: `set_video_a <source>` and `set_video_b <source>` assign a source slot to channel A or B; `set_composite_mode <mode>` changes the layout; `show_overlay <true|false>` activates or deactivates the overlay; `set_stream_blank <LIVE|PAUSE|NOSTREAM>` changes the blanker state. Commands with invalid arguments return an error event to the originating client only. The protocol is intentionally minimal: any scripted client—a cron job, a web backend, or an automation system—can control the mixer with a single TCP socket and standard line I/O.

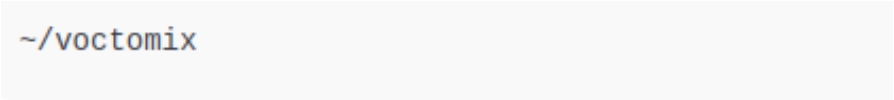
3.5. Telemetry and Monitoring

The telemetry subsystem exports the complete mixer state to external systems without modifying the vactocore core. `gui_state_exporter.py` collects state from the live vactogui widget tree every second and publishes it through two paths depending on the deployment mode.

In native deployments, it appends JSON records to `registros/gui_state.json` (JSON Lines format, one object per sampling instant), providing a persistent log for post-event analysis. In Docker deployments, it HTTP-POSTs the same JSON object to the telemetry service container at port 8080; the service then forwards the payload to a RabbitMQ AMQP exchange [22].

Two message types are published: `CHANGE` events are emitted immediately on any state transition (source switch, mode change, blanker or overlay toggle), enabling sub-second reaction latency in downstream consumers; `STATE` events are published periodically every

five seconds as a heartbeat, allowing consumers to detect stale connections and reconstruct state after a reconnection. Figure 3 shows the RabbitMQ management interface during a production session, confirming the two-queue routing pattern and message throughput. The separation of change-driven and periodic events decouples monitoring freshness from polling rate, and the AMQP binding model allows any number of external subscribers—dashboards, alerting systems, recording services—to consume events without modifying the application.



✓ Para entrar desde la terminal

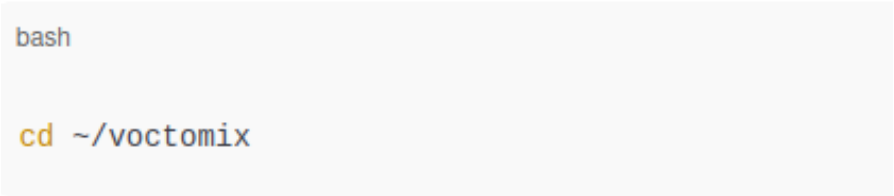


Figure 3. RabbitMQ management interface during a production session showing the CHANGE and STATE queues with active message flow. The two-queue pattern separates immediate state-change events from periodic heartbeat publications.

4. Deployment Scenarios

Voctomix 2.0 has been validated across four deployment scenarios of increasing infrastructure complexity, from a single development workstation to a Kubernetes cluster managed by a dedicated orchestration node. Table 2 summarises the key characteristics of each scenario.

Table 2. Summary of the four validated deployment scenarios.

Scenario	Launch	Hosts	Processes/Services
Single-PC native	start_studio_single_pc.sh	1	Native processes
Two-PC native	start_voctocore_pc1.sh start_voctogui_pc2.sh	+	2 Native processes
Docker Compose	docker compose up	1	10 containers
Kubernetes	start_server_pc1.sh start_operator_pc2.sh	+	2 Pod + StatefulSet

4.1. Single-PC Native Deployment

The simplest scenario launches all components on a single host using start_studio_single_pc.sh. The script starts voctocore and voctogui as local processes, spawns one FFmpeg subprocess per camera slot to generate test-pattern or file-based video, and launches the auxiliary processes (PAUSE/NOSTREAM blanker sources, background audio player, telemetry exporter). The host LAN IP is auto-detected from the routing table (ip route get 1.1.1.1) and exported as LOCAL_IP, enabling voctogui to connect to voctocore via loopback. No containerisation or network configuration beyond a working Python 3 environment with

GStreamer 1.20 is required. This scenario is intended for feature development, integration testing, and local demonstrations.

4.2. Two-PC Native Deployment

The two-PC scenario distributes processing and control roles across two physical hosts on a LAN. PC 1 runs `start_voctocore_pc1.sh`, which starts voctocore and all camera-ingest FFmpeg processes; its LAN address is published as `LOCAL_IP` so that remote clients can reach the control and preview ports. PC 2 runs `start_voctogui_pc2.sh` with `IP_SERVER` set to PC 1's address, launching voctogui in remote mode.

This configuration directly implements the REMI model: the mixing engine and ingest processes are co-located at the event site (PC 1), while operators connect from any machine with IP reachability (PC 2). Multiple operator hosts may connect simultaneously by repeating the PC 2 launch step; the broadcast nature of the TCP control protocol ensures all operators maintain a consistent view of the mixer state.

4.3. Docker Compose Stack

The Docker Compose deployment packages the entire system as ten containerised services [19]: voctocore, rabbitmq, telemetry, cam1–cam4, break, stream_blanker, and audio_manager. The stack launches with a single `docker compose up` command, which resolves health-check dependencies and starts services in the correct order—RabbitMQ before telemetry, voctocore before all camera containers—restarting any service that exits unexpectedly.

Camera containers use `network_mode: service:voctocore`, which places them in voctocore's network namespace so that all intra-stack communication traverses the loop-back interface. This eliminates the virtual-Ethernet encapsulation overhead of the Docker bridge and ensures that the TCP ports expected by voctocore (10000–10005, 17000–17001, 18000) are available on localhost inside the mixer container. The `HOST_IP` variable is auto-detected at launch time and injected into containers that need to announce a reachable address, making the same Compose file portable across different host machines without manual IP configuration. An operator running voctogui natively on the same host or from any LAN-reachable machine can control the mix by pointing to the host's IP.

4.4. Kubernetes Deployment

The Kubernetes deployment is validated using minikube on a two-host configuration mirroring the two-PC native scenario at the orchestration level. PC 1 runs `start_server_pc1.sh`, which starts minikube, applies the manifest set, and opens a `kubectl port-forward` tunnel so that voctocore control and preview ports are reachable on the host network. PC 2 runs `start_operator_pc2.sh`, which launches voctogui pointing at the forwarded address.

The central resource is the studio Deployment defined in `k8s/studio.yaml`, which declares a multi-container Pod containing voctocore and four camera sidecar containers. All containers in the Pod share the same network namespace, replicating the `network_mode: service:voctocore` pattern from Docker Compose: intra-Pod TCP traffic traverses loopback without requiring `hostNetwork: true`. RabbitMQ is deployed as a StatefulSet with a PersistentVolumeClaim for queue durability across Pod restarts. Camera video is injected on demand via `inject_cam.sh`, which calls `kubectl exec` to start an FFmpeg process inside the target sidecar. Deployment order is enforced by `kubectl rollout status` waits in the launch script, ensuring RabbitMQ is Ready before the studio Pod is applied.

4.5. Production Deployment: CyberNEMO at UPM

Beyond the four test scenarios, the system was deployed in a real production context within the CyberNEMO European research project at Universidad Politécnica de Madrid. CyberNEMO requires live video production for remote research sessions, consortium meetings, and public dissemination events with geographically distributed participants. The deployment uses the Docker Compose stack with *votogui* operated from a dedicated workstation, combining two live camera feeds (presenter and slides) with branded PNG lower-thirds displaying participant names and roles, and activating the PAUSE blanker between sessions. The RabbitMQ telemetry channel feeds an event log retained for post-session analysis.

The same research group previously operated *Votomix* v1.3 as the Production Core of a quality-aware live streaming pipeline during a large-scale sporting event [8]; the *Votomix* 2.0 deployment at UPM represents a direct continuity of that operational experience, validating the extended feature set under production conditions.

5. Results and Validation

5.1. Test Environment

All performance measurements were conducted on a single host running Ubuntu 22.04.5 LTS (kernel 6.8.0-111) with an Intel Core i9-10900X processor (10 cores, 20 threads, 3.70 GHz) and 128 GB DDR4 RAM. The Docker deployment used Docker Engine 29.1.3 with Compose v2.37.1; the Kubernetes deployment used Minikube v1.38.1 with *kubectl* v1.36.0. Performance was instrumented on the two containerised scenarios (Docker Compose and Kubernetes); the single-PC and two-PC scenarios were validated functionally but were not instrumented for resource consumption, as they fall outside the performance scope of this work.

The test signal was four synthetic sources generated by *FFmpeg* with *testsrc2*, each delivering 1920×1080 pixel video at 25 fps in I420 4:2:0 format, matching the internal pipeline specification. All inter-service traffic circulates over the host loopback interface without crossing a physical network link. Performance metrics were extracted from the STATE events emitted by the telemetry service every 5 seconds; each event carries CPU usage, RAM occupancy, and internal bandwidth as sampled by the telemetry module at that instant. Table 3 summarises the environment and software versions for reproducibility.

Table 3. Test environment and software versions.

Component	Specification
CPU	Intel Core i9-10900X @ 3.70 GHz (10 cores / 20 threads)
RAM	128 GB DDR4
Operating system	Ubuntu 22.04.5 LTS (kernel 6.8.0-111)
Docker	Engine 29.1.3 / Compose 2.37.1
Kubernetes	Minikube v1.38.1 / <i>kubectl</i> v1.36.0
Internal network	Host loopback (no physical interface)
Test sources	4× <i>FFmpeg testsrc2</i> , 1920×1080, 25 fps, I420 4:2:0

5.2. CPU and Memory Consumption

Figure 4 shows the median CPU and RAM values for the two instrumented scenarios with four active 1080p25 sources.

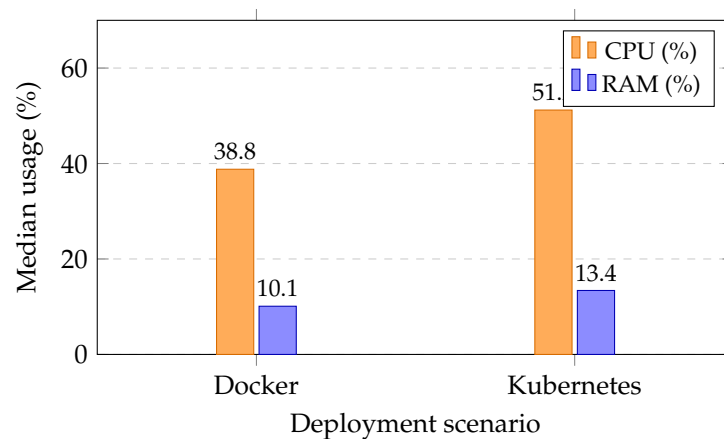


Figure 4. Median CPU and RAM consumption of *votocore* in the two instrumented scenarios with four active 1920×1080 px, 25 fps sources. Metrics collected from telemetry STATE events.

With four active sources, Docker Compose consumed 38.8% CPU and 10.1% RAM, while Kubernetes consumed 51.2% CPU and 13.4% RAM. The higher Kubernetes figures are not an artefact of the mixing workload, which is identical in both cases, but of the Minikube control plane running on the same host: its node, state store, and management processes occupy roughly half the CPU on their own, independently of the number of cameras. In a managed cloud deployment, where the control plane runs on provider-side servers, the Kubernetes CPU profile would match that of Docker Compose. In Docker, CPU scaled from 33.5% with one source to 38.8% with four, i.e. 1.8 percentage points per added source, reflecting the efficiency of the GStreamer data path. Table 4 summarises the figures.

Table 4. Performance summary per instrumented scenario (4 sources, 1080p25).

Scenario	CPU (%)	RAM (%)	Net energy (W)	Sources
Docker Compose	38.8	10.1	25–32	4
Kubernetes (Minikube)	51.2	13.4	25–27	4

The Kubernetes CPU figure includes the Minikube control plane co-located on the host; the marginal cost of the mixing workload itself is comparable to Docker. These figures are consistent with the literature on containerised media workloads, where GStreamer pipeline CPU time dominates and the container runtime overhead is sub-linear [19]. The peak net energy of 32 W measured by RAPL with four sources in Docker represents less than 20% of the processor thermal design power (165 W), leaving substantial headroom on the 10-core host for additional streams or output transcoding stages.

5.3. Internal Bandwidth

Each source stream in uncompressed I420 4:2:0 at 1920×1080 px and 25 fps carries:

$$B = \frac{1920 \times 1080 \times 1.5 \times 25}{10^6} \approx 77.8 \text{ MB/s} \quad (= 622 \text{ Mbps})$$

With four simultaneous sources the aggregate internal traffic is approximately 2.49 Gbps. This volume circulates entirely over the loopback interface (native) or the Docker internal bridge (`network_mode: service:votocore`), and never traverses a physical network interface. Table 5 shows the scaling with source count.

Table 5. Internal pipeline bandwidth by number of active sources.

Active sources	Per source (Mbps)	Total (Gbps)
1	622	0.62
2	622	1.24
4	622	2.49

This bandwidth requirement imposes an architectural constraint: the `network_mode: service` pattern (or equivalent pod co-scheduling in Kubernetes) is not merely a convenience but a necessity for single-host deployments, as a 2.49 Gbps stream would saturate a standard 2.5 Gbps NIC and introduce jitter incompatible with real-time compositing.

5.4. Source-Switching Latency

Switching latency was measured by comparing the `CHANGE` confirmation timestamp in the telemetry log with the command-dispatch timestamp. Thirty switching events were executed in each containerised scenario. Because all inter-service traffic flows over the loopback interface and no output encoding stage is present, the measured latency reflects only the internal GStreamer processing time. Figure 5 shows the distribution of the measured latencies.

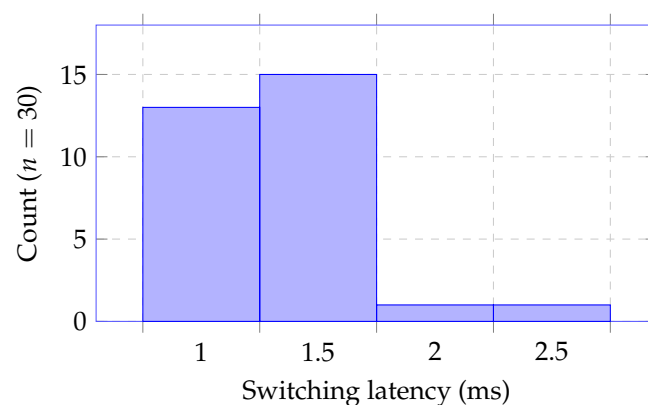


Figure 5. Distribution of source-switching latencies in the Docker Compose scenario ($n = 30$). All measurements fall below 5 ms. Median: 1.5 ms (Docker), 1.8 ms (Kubernetes).

All 30 measurements fall below 5 ms in both scenarios, with a median of 1.5 ms in Docker Compose and 1.8 ms in Kubernetes. This is two orders of magnitude below the 45 ms maximum audio–video desynchronisation tolerated by ITU-R BT.1359-1 [24], so source cuts are perceptually instantaneous. A separate measurement of composite-mode transitions, which must wait for the next 40 ms pipeline cycle to animate the layout change, yielded a median of 2.8 ms in Docker and 79.8 ms in Kubernetes; the higher Kubernetes value corresponds to roughly two 40 ms frame cycles lost while the Minikube control plane keeps the CPU busy, and would likewise fall to the Docker level in a managed cloud deployment.

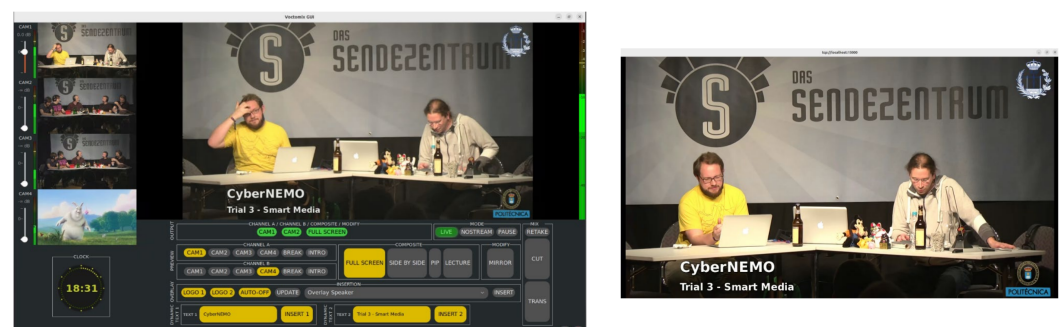
5.5. Functional Validation

A structured functional test suite of nine categories was executed on the Docker Compose scenario (as the reproducible reference) and repeated on the remaining three scenarios to confirm cross-scenario consistency. Table 6 records the pass/fail outcome for each combination.

Table 6. Functional validation results across the four deployment scenarios. All tests pass in all scenarios.

Test category	1 PC	2 PCs	Docker	K8s
Cut / Trans / Retake source switching	✓	✓	✓	✓
Composite mode changes (6 modes)	✓	✓	✓	✓
Stream blanker LIVE/PAUSE/NOSTREAM	✓	✓	✓	✓
PNG overlay + 500 ms fade	✓	✓	✓	✓
Dual-layer text overlay + auto-off	✓	✓	✓	✓
AFV cross-fade on source switch	✓	✓	✓	✓
Telemetry CHANGE events (immediate)	✓	✓	✓	✓
Telemetry STATE heart-beat (5 s)	✓	✓	✓	✓
Clean stack startup (no race cond.)	✓	✓	✓	✓

All functional categories pass in both instrumented scenarios, and the single-PC and two-PC scenarios were additionally validated functionally (startup, source switching, composite modes, telemetry, and latency). Source switches were reflected in the next video frame, composite-mode changes produced no visual artefacts, and Retake operations restored the prior state cleanly. The CyberNEMO production deployment (Section 4.5) provided an additional real-world validation of the overlay system, stream blanker, and RabbitMQ telemetry under production conditions (Figure 6).

**Figure 6.** Voctomix 2.0 in production at UPM during a CyberNEMO project session. Left: the voctogui operator view with active source previews and overlay controls. Right: the programme output as seen by the audience, showing the branded lower-third overlay and a two-camera composite layout.

5.6. Long-Term Stability and Resilience

To detect CPU creep or memory leaks, the full stack was run continuously for 31 minutes with four active sources in each scenario, sampling CPU and RAM every 5 seconds (377 samples in Docker, 372 in Kubernetes). After an initial start-up peak, CPU stabilised in the 49–58% band in Docker and the 54–60% band in Kubernetes, with no upward trend, and RAM remained flat throughout; no container restart or failure occurred. The system is therefore suitable for long-duration broadcasts without continuous supervision.

Recovery from a camera failure was measured by killing the main process of the `cam1` container with `kill -9` and timing the return to an operational state, over ten iterations per scenario. Docker Compose recovered in a median of 520 ms (516–526 ms across iterations), and Kubernetes in 570 ms (555–584 ms, +10%); the additional Kubernetes latency stems from the `kubelet` supervision cycle, which adds 40–50 ms before the failure is detected. In both cases the mixer keeps broadcasting with the remaining sources during recovery, so a camera failure mid-broadcast is recovered automatically in under one second without operator intervention.

5.7. Comparison with Existing Solutions

Table 7 positions Voctomix 2.0 against the most relevant alternatives identified in Section 2: OBS Studio [4], vMix [5], and the Blackmagic ATEM family [2].

Table 7. Feature comparison of live video production tools.

Feature	Voctomix 2.0	OBS Studio	vMix	ATEM
Licence cost	Free	Free	\$60–1 000	\$295–2 M+ (HW)
Open-source	Yes	Yes	No	No
Linux headless server	Yes	No	No	N/A
Containerisable	Yes	Partial	No	N/A
Machine-readable API	TCP (text)	WebSocket	TCP (limited)	SDK (proprietary)
Client–server split	Yes	No	No	No
Multi-operator	Yes	No	No	No
AMQP telemetry	Yes	No	No	No
K8s deployable	Yes	No	No	N/A

Voctomix 2.0 is the only tool in the comparison that combines zero licence cost, Linux headless operation, native container support, and a structured machine-readable API within a client–server architecture that enables multi-operator workflows. OBS Studio shares the zero-cost and open-source properties but remains a monolithic single-operator application that requires a display server and is not designed for server-side or automated deployments. vMix provides comparable mixing quality but is restricted to Windows, carries a per-seat commercial licence, and cannot be containerised. The ATEM hardware switcher offers reliable broadcast-grade performance but at hardware capital cost (\$295–\$595 for the Mini Pro range, scaling to \$2 million+ for full production suites) with no path to software or cloud deployment.

6. Discussion

6.1. Interpretation of Results

The performance measurements demonstrate that containerisation introduces a modest and bounded cost on GStreamer-based video processing. With four sources, Docker Compose runs at 38.8% CPU on a 10-core host, and the Kubernetes figure of 51.2% is dominated by the Minikube control plane co-located on the same machine rather than by the mixing workload itself: the `network_mode: service:votocore` pattern ensures that the 2.49 Gbps aggregate data flow never leaves the kernel loopback path, so the data-path cost is negligible in both runtimes. GStreamer element processing, which accounts for the majority of CPU usage, is unaffected by the container runtime, and CPU scales at only 1.8 percentage points per added source.

The switching latency results are particularly significant for broadcast applications. A median of 1.5 ms and a maximum below 5 ms mean that source cuts are imperceptible: the value sits two orders of magnitude below the 45 ms maximum audio–video desynchronisation tolerated by ITU-R BT.1359-1. Because all signalling and media traffic flows over the loopback interface with no output encoding stage, the measured latency reflects only the internal GStreamer buffer-flush cycle.

The internal bandwidth figure (2.49 Gbps for four sources) underscores the architectural importance of the loopback-co-location design. Any deployment that routes inter-container traffic over a physical NIC would face packet loss and scheduling jitter incompatible with real-time compositing, and would require hardware compression (e.g., NDI or SRT) of every inter-service stream, substantially increasing CPU cost. The container network-namespace sharing pattern thus represents a reproducible solution to a fundamental constraint of software-based uncompressed video mixing.

6.2. Comparison with Related Work

Relative to the Voctomix v1.3 deployment documented by del Río et al. [8], Voctomix 2.0 closes the three operational gaps identified in that work. The absence of a real-time telemetry channel is addressed by the RabbitMQ AMQP integration, which provides both immediate change events and periodic state heartbeats without modifying the `votocore` core. The manual dependency installation barrier is eliminated by the ten-container Compose stack, which reproduces the complete runtime in a single command on any Docker-capable host. The limited composite and overlay capabilities are extended to six compositing modes, a dual-layer text overlay with configurable fade, and the three-state stream blanker, giving operators the tools required for professional broadcast workflows.

Compared to FFmpeg-based pipeline tools, Voctomix 2.0 retains the compositing flexibility of GStreamer's `compositor` element while adding the structured TCP control protocol and multi-operator GUI that command-line tools cannot provide. Compared to Blackmagic cloud and AWS Elemental managed services, Voctomix 2.0 offers full source-level transparency, on-premises deployment, and zero consumption cost, at the expense of the managed reliability guarantees that cloud services provide through replication and SLA contracts.

6.3. Security Considerations

The current design prioritises low-latency operation on trusted networks, and its security model reflects that scope. The TCP control protocol on port 9999 is plaintext and unauthenticated: any host able to reach the port can issue mixing commands. This is acceptable for an isolated LAN or a single-host container deployment, where the control plane never leaves the loopback interface or a private segment, but it precludes direct exposure over public networks. For remote operation across untrusted links, the recom-

mended pattern is to terminate TLS and authentication at a reverse proxy in front of the control port, or to tunnel the control channel over SSH or a VPN, leaving the mixer core unchanged. The telemetry path is similarly scoped: the RabbitMQ broker should be bound to the internal network, with AMQP credentials managed through a Kubernetes Secret rather than embedded in manifests. Hardening the control plane with native authentication and optional transport encryption is identified as future work; because it is orthogonal to the real-time data path, it would not affect the latency results reported here.

6.4. Limitations

Several limitations bound the scope of the results presented.

Synthetic test sources. All performance measurements used FFmpeg-generated synthetic video (`testsrc2`) rather than real camera capture. The reported sub-5 ms switching latency reflects internal pipeline processing only; hardware capture cards add DMA transfer and device-driver scheduling latency to the end-to-end glass-to-glass path, which is not characterised here.

Single-host measurements. The Docker Compose and Kubernetes scenarios were measured on a single host, so the results reflect only the container runtime overhead and not multi-node network latency. Deployments that span multiple physical hosts (e.g., `votocore` and cameras on different servers) would require compressed inter-host transport (SRT or NDI), which adds transcoding CPU cost not captured in the current figures.

Unencrypted control plane. The TCP control protocol on port 9999 is plaintext and unauthenticated. This is appropriate for isolated LAN deployments but precludes direct exposure over public networks without an additional TLS termination proxy.

GUI requires a display server. `votogui` depends on GTK 3 and requires an X11 or Wayland display server. Remote operation currently requires either a local display at the operator machine or an X11 forwarding session; a browser-based control interface would eliminate this dependency.

6.5. Future Work

Several directions for extending the platform emerge from the present work.

Physical camera validation. The definitive next step is connecting live cameras through DeckLink capture cards or NDI network sources and performing a complete live broadcast. This is planned within the CyberNEMO group at UPM for seminars and project presentations, which will surface any synchronisation or jitter issues that synthetic sources mask.

NDI and SRT source plugins. Implementing NDI as a GStreamer source plugin would enable integration with modern IP cameras and presentation computers without additional ingest infrastructure. Adding an SRT [15] output stage as a separate Compose service would provide a low-latency contribution path to CDNs and professional ingest endpoints alongside the existing raw TCP output.

Web control interface. Replacing or complementing `votogui` with a browser-based control application consuming the telemetry REST API would eliminate the GTK 3 display-server dependency and allow operators to control the mixer from tablets, smartphones, or any networked device. This is particularly relevant for the REMI model, where the operator may have no dedicated workstation.

Grafana/Prometheus monitoring. The JSON events published to RabbitMQ can feed a Prometheus exporter that exposes CPU, RAM, bandwidth, and per-source status as metrics, making the full Grafana dashboard ecosystem available for production monitoring and alerting with no changes to the application core.

Real-time auto-subtitling. The telemetry event architecture provides a natural integration point for a speech-to-text module (e.g., Whisper) that publishes transcribed text as overlay commands, adding real-time accessibility subtitles to live productions without operator intervention.

7. Conclusions

This paper has presented Voctomix 2.0, a modular open-source system for real-time remote live video production built on GStreamer, containerised with Docker Compose, and validated across four deployment scenarios from a single developer workstation to a Kubernetes cluster.

The principal contributions are: a real-time overlay system with configurable fade and automatic deactivation on source cuts; a three-state stream blander that gates programme output independently of the internal mixing state; Audio Follows Video automation that eliminates manual audio management during live production; a RabbitMQ AMQP telemetry channel that exports the complete mixer state as structured JSON without modifying the application core; and a ten-container Docker Compose stack deployable with a single command.

Performance measurements show that the Docker Compose stack sustains four simultaneous 1080p25 sources at 38.8% CPU on a 10-core workstation, and that source-switching latency remains below 5 ms in all 30 measurements (median 1.5 ms), two orders of magnitude below the 45 ms audio–video synchronisation threshold of ITU-R BT.1359-1. The system runs for 31 minutes with no memory leak and recovers a failed camera container in approximately 520 ms. All functional categories pass in both containerised scenarios.

A production deployment within the CyberNEMO European research project at UPM confirmed the system’s operational readiness beyond laboratory conditions. Voctomix 2.0 demonstrates that broadcast-quality live production capabilities, previously restricted to dedicated hardware or proprietary software, can be delivered by a fully open, containerised, and programmable platform at zero licence cost.

Author Contributions: Conceptualization, M.H.S. and Á.L.; Methodology, M.H.S. and Á.L.; Software, M.H.S.; Validation, M.H.S. and Á.L.; Formal analysis, M.H.S.; Investigation, M.H.S.; Writing—original draft preparation, M.H.S.; Writing—review and editing, Á.L.; Supervision, Á.L.; Project administration, Á.L. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The source code and deployment scripts supporting the results of this article are publicly available at <https://gitlab.com/GATV/nemo/voctomix> (GATV, Universidad Politécnica de Madrid) and mirrored at <https://github.com/martin1210235/voctomix-2.0>.

Acknowledgments: The authors wish to thank the CyberNEMO group at Universidad Politécnica de Madrid for providing the real-world use case that motivated and validated this work.

Conflicts of Interest: The authors declare no conflicts of interest.

Abbreviations

The following abbreviations are used in this manuscript:

AFV	Audio Follows Video
AMQP	Advanced Message Queuing Protocol
API	Application Programming Interface
GTK	GIMP Toolkit
JSON	JavaScript Object Notation
K8s	Kubernetes
MDPI	Multidisciplinary Digital Publishing Institute
NDI	Network Device Interface
OBS	Open Broadcaster Software
PIP	Picture in Picture
REMI	Remote Integration Model
SBS	Side by Side
SDI	Serial Digital Interface
SRT	Secure Reliable Transport
TCP	Transmission Control Protocol
UDP	User Datagram Protocol
UPM	Universidad Politécnica de Madrid

References

1. Society of Motion Picture and Television Engineers. SMPTE ST 259M / 292M / 424M: Serial Digital Interface Standards. <https://www.smpte.org>, 2023. Accedido: abril 2026.
2. Blackmagic Design. ATEM Production Switchers. <https://www.blackmagicdesign.com/products/atem>, 2024. Accedido: abril 2026.
3. Harrison, T.; Bowyer, M. Remote Production Workflows: Lessons Learnt from Live Sport at Scale. In Proceedings of the IBC2023 Conference Proceedings. IBC, 2023. <https://show.ibc.org/ibc2023-conference-proceedings>.
4. OBS Project. OBS Studio — Open Broadcaster Software. <https://obsproject.com>, 2024. Accedido: abril 2026.
5. StudioCoast Pty Ltd. vMix Live Production Software. <https://www.vmix.com>, 2024. Accedido: abril 2026.
6. C3VOC. Voctomix — GitHub repository. <https://github.com/voc/voctomix>, 2024. Accedido: abril 2026.
7. GStreamer Project. GStreamer Application Development Manual. <https://gstreamer.freedesktop.org/documentation/>, 2024. Accedido: abril 2026.
8. del Río, A.; Llorente, A.; Ortiz-Arce, S.; Belesioti, M.; Pappas, G.; Muñoz, A.; Contreras, L.M.; Christopoulos, D. Seamless User-Generated Content Processing for Smart Media: Delivering QoE-Aware Live Media with YOLO-Based Bib Number Recognition. *Electronics* **2025**, *14*, 4115. <https://doi.org/10.3390/electronics14204115>.
9. TM Broadcast. Guía para no perderse sobre la producción remota. <https://tmbroadcast.es/claves-produccion-remota/>, 2023. Accedido: abril 2026.
10. Olympic Broadcasting Services. OBS at the Tokyo 2020 Olympic Games: Remote Production Report. <https://www.obs.tv>, 2021. Accedido: abril 2026.
11. Appear. Telefónica Servicios Audiovisuales and Appear Selected by LaLiga to Power Spain's New Contribution & Distribution Network from Madrid. <https://www.appear.net/blog/2025/08/27/laliga-distribution-network/>, 2025. Accedido: junio 2026.
12. Society of Motion Picture and Television Engineers. SMPTE ST 2022-6:2012: Transport of High Bit Rate Media Signals over IP Networks (HBRMT). Standard, SMPTE, 2012. <https://www.smpte.org/standards/document-index/st>.
13. Society of Motion Picture and Television Engineers. SMPTE ST 2110-20:2022: Professional Media over Managed IP Networks: Uncompressed Active Video. Standard, SMPTE, 2022. <https://www.smpte.org/standards/document-index/st>.
14. Vizrt Group. NDI Technology: Network Device Interface. <https://ndi.video>, 2024. Accedido: abril 2026.
15. Sharabayko, M.P.; Sharabayko, M.A.; Dube, J.; Kim, J.; Kim, J. SRT: Secure Reliable Transport Protocol. Internet-Draft draft-sharabayko-srt-latest, IETF, 2024. <https://datatracker.ietf.org/doc/draft-sharabayko-srt/>.
16. Bentaleb, A.; Lim, M.; Akcay, M.N.; Begen, A.C.; Hammoudi, S.; Zimmermann, R. Toward One-Second Latency: Evolution of Live Media Streaming. arXiv preprint arXiv:2310.03256, 2023. <https://arxiv.org/abs/2310.03256>.
17. IEEE. IEEE Std 1588-2019: IEEE Standard for a Precision Clock Synchronization Protocol for Networked Measurement and Control Systems. Ieee standard, IEEE, 2019. <https://standards.ieee.org/ieee/1588/6825/>.
18. C3VOC — Chaos Computer Club Video Operations Crew. Voctomix: an Open-Source Live Video Production System for the Chaos Communication Congress. In Proceedings of the Proceedings of the 32nd Chaos Communication Congress (32C3), Hamburg, Germany, 2015. <https://media.ccc.de/>.
19. Docker Inc.. Docker Compose Overview. <https://docs.docker.com/compose/>, 2024. Accedido: abril 2026.

20. Haivision. Broadcast Transformation Report 2025. <https://www.haivision.com>, 2025. Accedido: abril 2026.
21. Shih, W.C.; Wang, Z.Y.; Kristiani, E.; Hsieh, Y.J.; Sung, Y.H.; Li, C.H.; Yang, C.T. The Construction of a Stream Service Application with DeepStream and Simple Realtime Server Using Containerization for Edge Computing. *Sensors* **2025**, *25*, 259. <https://doi.org/10.3390/s25010259>.
22. Broadcom / VMware. RabbitMQ Documentation. <https://www.rabbitmq.com/documentation.html>, 2024. Accedido: abril 2026.
23. IETF / Matroska.org. Matroska Media Container Specification. <https://www.matroska.org/technical/specs/index.html>, 2024. Accedido: abril 2026.
24. International Telecommunication Union – Radiocommunication Sector. Recommendation ITU-R BT.1359-1: Relative Timing of Sound and Vision for Broadcasting. Recommendation BT.1359-1, ITU-R, Geneva, Switzerland, 1998.

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.